

A Python-Based File Integrity Monitoring Host-Based Intrusion Detection System

Technical Design and Implementation

Charlie Cleere

2026

A Python-Based File Integrity Monitoring Host-Based Intrusion Detection System

Introduction

A File Integrity Monitoring (FIM) Host-Based Intrusion Detection System (HIDS) is a security tool that detects file changes on a single computer (host). This paper presents a FIM HIDS written in Python. This Python-based FIM HIDS monitors specified directories, detects new, deleted, and modified files, and logs these changes in a log file. It is designed to be a cross-platform system and run periodically using a scheduler such as cron. The primary purpose of the system is to aid in finding unauthorized file changes on a computer.

System Overview

This section provides an overview of the workflow of the Python-based FIM HIDS. When a scheduled run of the program begins, the first thing it does is get the following information: the directories it will monitor, the paths of the baseline and audit log files, and the file extensions and directories to exclude in the monitoring process.

Next, the system checks whether a valid baseline file exists. If the baseline file is missing or invalid, the FIM HIDS will create a new baseline file by scanning the monitored directories, getting the hashes, sizes, and last modified times of each file within them, and writing this information to a file. This scheduled run would end after that, and the next run would do the detecting.

On the other hand, when a valid baseline exists, the program prepares to do detection. It does this by retrieving each file's hash, size, and last modified time from the baseline file. Then, the system scans the monitored directories to get a current snapshot of the files and their metadata.

Now, since the FIM HIDS has a baseline and current snapshot, it can detect file changes. If the system sees any files in the current snapshot and not in the baseline, it knows that those files are new. Then it checks if any files are in the baseline but not in the current snapshot. If so, the program knows that those files have been deleted. Lastly, if a file's hash differs between the baseline and current snapshot, it is considered modified.

After that, if any changes were found, the system appends log entries for the affected files and their metadata to the audit log file.

Finally, if any file changes were detected, using the current snapshot, the program will write a new baseline to the baseline file. On the next scheduled run, it will now use an updated baseline for comparisons.

Implementation

This section describes the design choices and implementation details of the Python-based FIM HIDS. Python was the programming language the system was written in due to its simplicity and strong support for file and directory operations. Python also provides built-in libraries such as `os`, `json`, `hashlib`, and `datetime`, which were all used in this program.

Within the `os` library, the following functions are used in the system to work properly with files and directories in a way that is fit for a cross-platform FIM HIDS: `os.path.dirname`, `os.path.abspath`, `os.path.join`, `os.walk`, `os.path.expanduser`, `os.path.exists`, `os.path.getsize`, and `os.path.getmtime`. Of these, `os.walk` is one of the most significant. This tool is used to recursively scan directories, and it makes it easy to exclude certain files and directories in the scan.

The `hashlib` library's role in the program is to hash files using SHA-256. As an additional point, when reading files to hash them, instead of reading the whole file at once, which could lead to issues when a file is too large, the system reads the files in chunks (4 KB at a time).

Moreover, the json library is used to retrieve data from JSON files such as the configuration and baseline files and store them in Python dictionaries so the system can use the data. The configuration JSON file stores the paths of the directories that will be monitored, baseline and log file paths, and file extensions and directories paths to exclude. As a side note, this allows the FIM HIDS to be easily configured without changing the source code. The baseline JSON file stores the path of each file in it, along with its hash, last modified time, and size (in bytes). Converting this JSON file to a Python dictionary by using the json library, enables comparisons with the files and metadata from the current snapshot. In the system, the json library is also used to write Python dictionaries to the baseline JSON file in a structured format.

The last library used in the program—the datetime library—is used to produce timestamps for the log entries and error messages. An additional use of this library is to make the last modified times in the logs readable.

In addition, the log entries in the system's audit log file are structured in a way that allows them to be easily parsed. The entries are consistent and the fields are key-value pairs and separated by a space. It is worth noting that the most important information for each log entry—the timestamp, event type, and file path—are positioned in the first three fields in the entry so that they are the primary focus. After that, the file's size and last modification time, which provide further forensic information, are displayed. However, for a modified file event, it shows the before and after values of the size and last modified time.

To make the FIM HIDS more robust and reliable under real-world conditions, error handling was also added. When the program tries to load the configuration file, if the file is not found, the JSON formatting is invalid, or any of the required settings are missing, then the

program will print an error message not to the audit log file but to standard output (stdout). After that, it will safely stop execution. Furthermore, when the system checks the status of the baseline file, if the baseline file is missing or invalid, this error will be logged in the audit log file. Then, the program will try to write a new baseline file. If it fails in doing that (e.g., due to permission or filesystem issues), the program will print an error message to stdout and execution will stop. If the creation (or recreation if it was previously invalid) of the new baseline file is successful, the program will print a log entry to the audit log that it was created (or recreated).

More error handling is included when the FIM HIDS scans the directories to get each file's metadata. During the scan, when a file cannot be opened, read, or the metadata of it cannot be accessed (e.g., due to permission or filesystem issues), the program skips scanning that file and prints a warning to stdout to let the user know.

Lastly, to allow continuous monitoring, the program is designed to run periodically by means of a scheduler like cron. The system includes a separate log file—a cron log file—for if the program is run via cron. In this case, for optimal use, the error messages that go to stdout can be redirected to the cron log file for later review and troubleshooting of runtime errors and warnings.

Demonstration and Results

This section provides a demonstration of the Python-based FIM HIDS being tested, and an explanation of the results. The FIM HIDS was tested on a Linux computer, and was run at scheduled times—every five minutes—through a cron entry. For the testing environment, there were two directories for the program to monitor with subdirectories and files inside them. Files were also inside the subdirectories. Additionally, all the files in the testing environment ended with one of the following extensions: .txt, .log, or .tmp. The FIM HIDS's configuration file

included the paths of the two directories for the program to monitor, a baseline and audit log file path, a subdirectory path to exclude, and two file extensions to exclude (.log and .tmp).

During the first scheduled run of the system, a baseline file did not exist. So, to create a new baseline file, the system scanned the two monitored directories (while ignoring the excluded subdirectory and file extensions specified in the configuration file) to get the files within them and their metadata. Then it successfully created a baseline file with the name and location specified in the configuration file, and wrote this information to it. As shown in Figure 1, the detection of a missing baseline file and the creation of a baseline file was also logged in the audit log file. As expected, the first scheduled run ended after this.

```
2026-04-29T21:10:01 event=BASELINE_MISSING path=".../fim-hids/baseline.json"  
2026-04-29T21:10:01 event=BASELINE_CREATED path=".../fim-hids/baseline.json"
```

Figure 1: Audit log entries showing that the baseline file was missing and then created

After that, several changes were intentionally made to test the system. In the monitored directories, a new .txt file was created, a .txt file was deleted, and the text inside a .txt file was modified. Moreover, two more changes to files with the excluded extensions were made: a .tmp file was created and a .log file was deleted. All the above file changes occurred outside the excluded directory. Lastly, within the excluded directory, the text inside a .txt file was modified.

Then, on the second scheduled run, since at this time a valid baseline file existed, the program went right to loading the information from the baseline file. After that, it scanned the monitored directories to generate a current snapshot of the files and their metadata. The FIM HIDS compared the current snapshot against the baseline, and logged in the audit log file any file changes that were detected. As seen in Figure 2, the system successfully detected, categorized,

and logged the file changes that happened in the monitored directories, while appropriately ignoring the changes that happened to files with the excluded extensions (.log and .tmp) and within the excluded directory. The log entries were also displayed in a well-structured and easy to parse format, and included accurate sizes and last modified times of the affected files. (For the sake of space, however, the sizes and last modified times are not included in the log entries in Figure 2.)

```
2026-04-29T21:15:01 event=NEW path=".../fim-hids/test2/f.txt"  
2026-04-29T21:15:01 event=DELETED path=".../fim-hids/test2/e.txt"  
2026-04-29T21:15:01 event=MODIFIED path=".../fim-hids/test1/test4/b.txt"
```

Figure 2: Audit log entries showing detections of a new, a deleted, and a modified file

Next, at the end of this second scheduled run, the program updated the baseline file with the current snapshot. As desired, the run after then used the correct baseline.

Conclusion

The cross-platform Python-based FIM HIDS that can run at scheduled times accurately detects file changes through baseline comparison, and logs these changes along with additional metadata to a log file in a structured format. It was implemented with the following Python libraries: os, json, hashlib, and datetime. Moreover, the configuration file allows the FIM HIDS to be easily configured, and, with strong error checking, the system is reliable under real-world conditions. Finally, the results of the test show that overall the FIM HIDS is a robust security tool and that it would be helpful in finding unauthorized file changes on a computer.

GitHub Repository

<https://github.com/charliecleere/fim-hids>